KAVACH

GEOLOCATION & LOCATION-EVIDENCE VERIFICATION

Detailed Technical Design, Technology Stack, Agent Mapping, APIs, Data Model, Implementation Plan and Test Strategy

Project	KAVACH — AI-driven intelligence for smarter incident resolution
Primary backend	FastAPI + Python, with Pydantic request/response contracts
Location sources	Browser/device geolocation, image EXIF metadata, registered facility coordinates, server timestamps
Core principle	Location evidence supports a decision; it does not prove that a complaint or photo is genuine.
Prototype constraint	Keep inference and storage costs low; start with deterministic checks and mockable tests.

Prepared as a build-oriented guide for the KAVACH team. It distinguishes proposed design decisions from capabilities that must be validated during implementation.

Version 1.0 • 12 October 2026

Contents

Use this guide as the feature specification before implementation and task allocation.

1. Executive summary and goals
2. What geolocation can and cannot prove
3. End-to-end architecture
4. Technology stack and responsibility boundaries
5. Location evidence sources
6. Agent-by-agent mapping
7. Location verification rules and decision model
8. Data model and database fields
9. API contracts and example payloads
10. Implementation examples
11. Security, privacy and anti-spoofing limits
12. Testing and acceptance criteria
13. Repository layout and implementation sequence
14. What is in the prototype vs later releases
15. Glossary and references

> Reading rule: This is a proposed technical design based on the KAVACH workflow discussed so far. It is not evidence that these components already exist in the codebase.

1. Executive summary and goals

KAVACH needs location evidence at two important moments: when a citizen reports an incident and when a field worker claims to have resolved it. A shared Location Evidence Verification service should collect and compare available signals, produce an explainable result, and make that result available to the relevant agents.

The feature is not a single GPS check. It is a small, reusable subsystem that gathers different evidence sources and keeps their provenance separate.

Goals

- Help determine whether a complaint's submission location is consistent with its reported incident location.
- Help determine whether a worker's completion evidence is consistent with the assigned facility location.
- Read GPS metadata from uploaded images when present, without assuming it exists or is trustworthy.
- Provide explainable outcomes: CONSISTENT, MISMATCH, INCONCLUSIVE, and optionally REVIEW_REQUIRED.
- Preserve source, accuracy, timestamps and reasons so supervisors can audit decisions.
- Support later mapping, dispatch optimization, hotspot analysis and recurrence prediction.

Non-goals for the first version

- Do not claim to prove a person is physically present or that a complaint is truthful.
- Do not automatically punish a citizen or worker solely because coordinates differ.
- Do not require every citizen to be physically at the incident; remote reporting can be legitimate.
- Do not use Gemini/LLM output as the authority for coordinate math or geofence decisions.
- Do not implement continuous background tracking. Capture only the location needed for the current report/task.

Recommended decision: accept complaints from anywhere, ask for device location with clear consent, and use location consistency as a trust signal. For workers, location can be a stronger task-evidence requirement if operational policy allows it, but there must still be an exception path.

2. What geolocation can and cannot prove

Latitude and longitude identify a point on Earth. They do not identify the person holding the device, establish the truth of a report, or guarantee that a photo was taken at that point.

Signal	Can support	Cannot prove alone
Device geolocation	Where the device reports it is at submission time	That the reporter is honest or that the incident exists
Image EXIF GPS	A location value recorded in photo metadata	That metadata is original, untampered, current or authentic
Registered facility GPS	The expected coordinates of a toilet/bin/site	That the facility pin is correct or precise enough for every nearby facility
Server receipt timestamp	When the server received the request	When the image was actually captured
Visual AI result	Whether image content appears consistent with a claimed issue or cleanup	Exact physical location or time without additional evidence
Repeated incident history	Recurring patterns and geographic hotspots	That an individual complaint is false

Use evidence fusion rather than a binary 'GPS = true/false' rule. A mismatch should mean 'needs review' unless a documented policy defines a narrower operational action.

3. End-to-end architecture

The UI obtains device location through the browser or native app permission flow. FastAPI validates the request and orchestrates the feature. A deterministic Python service compares coordinates and accuracy. Pillow inspects image metadata. The agents consume the result but do not replace these lower-level services.

Layer	Component	Responsibility
User interface	React / Browser Geolocation API	Request consent; capture current coordinates, accuracy and acquisition time; show failure/permission states
HTTP/API	FastAPI	Receive complaint or worker submission; enforce auth, size limits and validation; return API response
Contracts	Pydantic	Validate types, coordinate ranges, enums and request structure
Media inspection	Pillow	Read supported image EXIF fields and GPS IFD when present
Core logic	Python Location Verification Service	Normalize coordinates; calculate distance; account for accuracy; assign status and reason codes
Workflow	LangGraph (if used in project)	Route results through Incident Intelligence / Supervisor / Dispatch / Verification stages
AI	Gemini API	Interpret complaint text/image and summarize context; never be sole source of location truth
Persistence	PostgreSQL	Store incidents, facility coordinates, evidence records, checks and audit history
Spatial querying (optional)	PostGIS	Efficient radius searches and nearest-facility queries when mapping scale requires it
Observability	Python logging + structured event IDs	Record decisions and failures without leaking exact coordinates or sensitive EXIF into routine logs

Important: Browser geolocation normally requires HTTPS in deployed environments and explicit user permission. localhost is generally suitable for local development. The API can return an error, be denied, or provide inaccurate coordinates. [1]

4. Technology stack and why each tool is used

Technology	Use it for	Do not use it for
React (existing frontend plan)	UI action to request location, explain permission, show map pin and error states	Proving that submitted coordinates have not been manipulated
navigator.geolocation	Current device position, accuracy in metres and timestamp	Silent collection without permission or proof against spoofing
Pillow	Inspect EXIF metadata in an uploaded image; retrieve GPS IFD values	Calculating trustworthiness or deciding whether a complaint is fake
Python math / haversine	Approximate distance between coordinate pairs for prototype	Road travel distance or route planning
FastAPI	REST endpoints, upload handling, dependency injection and HTTP error responses	Business policy hidden inside a large endpoint function
Pydantic	Request and response schemas and validation	Authentication or proof that a client value is honest
PostgreSQL	Persistent relational records and audit trail	High-performance spatial index unless spatial features are added
PostGIS (optional)	Spatial indexes, radius filtering and nearby-site queries	Required dependency for the first small prototype
LangGraph (if adopted)	Workflow control, branching and state passing among agents	Coordinate calculations
Gemini API	Natural-language structuring and image content interpretation	GPS metadata extraction, distance calculations or unquestioned fraud decisions
pytest	Unit and integration tests, including spoofed/missing values	Runtime monitoring
HTTPS + auth/RBAC	Protect data in transit and restrict access to location evidence	Eliminating all device-side spoofing

5. Location evidence sources

KAVACH should preserve each source independently. Do not overwrite one source with another or collapse all coordinates into a single 'verified_location' field.

5.1 Device location at submission

- Collected in the React/browser layer using navigator.geolocation.getCurrentPosition(success, error, options).
- Useful values: latitude, longitude, accuracy (metres), position timestamp and collection method.
- Ask for location only when the user submits a complaint or worker task evidence; explain why it is requested.
- On permission denial, timeout or unsupported browser, allow a fallback flow and mark the evidence unavailable.
- Use HTTPS in deployed environments. The Geolocation API requires a secure context and user permission. [1][2]

5.2 Image EXIF GPS with Pillow

- Read EXIF from the uploaded original image bytes using Pillow's Image.open(...).
- Use image.getexif() and get_ifd(ExifTags.IFD.GPSInfo) to access the GPS IFD where present. [3]
- GPS values may be stored as degrees/minutes/seconds rational values and need conversion to decimal degrees.
- Read GPS latitude/longitude reference tags (N/S and E/W); apply negative sign for South and West.
- Check image format, byte size, dimensions, malformed metadata and decoding errors. Do not trust file extension alone.
- EXIF may be missing because of camera settings, messaging apps, image editing, screenshots or export pipelines. It can also be altered.

5.3 Registered facility location

- Each facility (toilet, public bin, sanitation zone, or other site) needs an admin-approved reference point.
- Store coordinates, facility identifier, source of the coordinates, who verified them and when.
- Validate the pin during setup. A bad reference pin creates false mismatch alerts.
- For nearby toilets in the same complex, GPS may not be enough to distinguish one facility from another; use a facility QR code, NFC tag, task-specific code or supervisor evidence as an additional signal.

5.4 Server time vs client/photo time

- server_received_at is generated by the backend/database and is the most dependable timestamp for receipt.
- Device location timestamp indicates when the browser says it measured the position; treat client data as untrusted.
- EXIF capture time is editable and may be absent or wrong. Store it as claimed capture time, not trusted proof.
- Record time deltas for review, but do not reject old images automatically without an explicit product rule.

6. Agent-by-agent mapping

Location is a shared capability, not an agent by itself. The verification service calculates and records evidence; agents use its outputs according to their responsibilities.

Agent / stage	Location-related role	Input	Output / decision
1. Incident Intelligence Agent	Use citizen submission location and EXIF metadata to enrich the structured incident; compare with the citizen's selected/pinned incident location; preserve source and confidence	Complaint text, selected incident pin, device location, optional image EXIF result	Structured incident location, source, missing/conflict flags, location consistency summary

Agent / stage	Location-related role	Input	Output / decision
2. Duplicate Detection Agent	Use normalized incident coordinates or facility ID as one duplicate signal, alongside text/category/image similarity	Incident ID, coordinates, category, text/image embeddings or hashes	Potential duplicates with spatial proximity; never merge solely because points are close
3. Supervisor Agent	Review location mismatch, weak accuracy, missing evidence or conflicting location sources; decide whether clarification is needed	Incident summary, location status, distance, accuracy, reasons, severity	Approve, request clarification, mark for manual review, or proceed
4. Sanitation Agent	Use confirmed facility/incident location to form the task and checklist; identify the correct facility	Incident/facility ID and approved coordinates	Task target location and facility-specific instructions
5. Dispatch Agent	Compare worker availability/location (if collected with consent and policy) with task coordinates; estimate geographic proximity	Task location, eligible workers, their available/authorized location data	Candidate worker ranking and estimated straight-line distance; road ETA needs routing data
6. Field worker execution	Capture location at task start/completion; upload before/after photos; show missing or inaccurate GPS state	Assigned task, device location, photos, status update	Evidence bundle tied to task ID and server receipt time
7. Resolution Verification Agent	Compare completion location with assigned facility; consume EXIF GPS and device GPS; combine with visual before/after analysis	Task target coordinates, device GPS, EXIF result, photos, task metadata	Location status plus visual verification result; pass/fail/review according to policy
Shared Location Verification Service	Pure, deterministic logic called by APIs/agents	Coordinates, accuracy, timestamps, target coordinates and source metadata	Distance, evidence status, reason codes, comparison details

No separate GPS agent is required for the first release. Keep geospatial math in normal Python service code; use LLM agents for language/image reasoning and workflow decisions where appropriate.

7. Location verification rules and decision model

A configurable rules engine should return an explainable result, not a hidden fraud score. Start with simple rules, log each reason, and calibrate thresholds using real field tests.

Status	Meaning	Typical handling
CONSISTENT	Available coordinates are close enough to the target under the configured rule and reported accuracy is acceptable	Continue to other validation checks; do not mark complaint automatically true
MISMATCH	Reported point is clearly outside the configured allowed radius, taking accuracy into account	Flag for clarification/review; don't automatically accuse the person
INCONCLUSIVE	Missing location, poor accuracy, conflicting sources, invalid timestamp or unsupported metadata	Use fallback evidence or manual review
NOT_CHECKED	Location collection was not requested or no target coordinate exists	Continue using non-location evidence
REVIEW_REQUIRED	Several signals conflict or risk rules are triggered	Supervisor reviews the reason codes and evidence

7.1 Distance

For a prototype, calculate great-circle distance with the haversine formula. This estimates straight-line distance; it does not calculate walking/driving distance. For many spatial queries or nearest-facility search, PostGIS geography functions such as ST_DWithin can perform radius checks in metres and use spatial indexes. [4]

7.2 Accuracy-aware comparison

If the browser reports accuracy of 80 m, a point that is 25 m from the facility should not be treated as a precise match or mismatch. The accuracy value is itself device-reported and not a guarantee. Store it and use it to make the outcome more cautious.

7.3 Example policy (illustrative only)

For a first test, a team might configure a target radius and an acceptable accuracy range per site type. These are product settings, not universal GPS truths. Do not hard-code a 10 m radius for every toilet: sites, devices, buildings and adjacent facilities differ.

7.4 Location-source conflict

- Device GPS near Toilet A; EXIF GPS far away: mark sources_conflict and review capture timing/context.
- Device GPS far away; no EXIF: remote report is possible; don't call it fake automatically.
- EXIF GPS near Toilet A; device GPS far away: image may be old, copied or taken earlier; investigate rather than automatically reject.
- Both sources near the target: location consistency is stronger, but sources can still be spoofed or the image may not show the claimed issue.
- Neither source available: continue with selected pin, photo, text, duplicate detection and human review.

8. Data model and database fields

Prefer an evidence table that can store multiple sources per complaint/task. This is more flexible than one wide table and preserves provenance.

Suggested tables

Table	Example fields	Why it exists
facilities	id, name, type, latitude, longitude, location_source, verified_by, verified_at	Approved reference location for each toilet/bin/site
incidents	id, reporter_id, description, category, selected_latitude, selected_longitude, facility_id, created_at	The complaint and claimed incident location
tasks	id, incident_id, assigned_worker_id, target_facility_id, status	Links field work to the correct target
location_evidence	id, subject_type, subject_id, source_type, latitude, longitude, accuracy_m, observed_at, received_at, metadata_json, trust_level	Stores device, EXIF, selected-pin or admin-reference evidence separately
location_checks	id, subject_type, subject_id, target_latitude, target_longitude, distance_m, status, reason_codes, rule_version, checked_at	Auditable result of a particular comparison
audit_events	id, actor_id, action, subject_id, event_time, safe_details_json	Supervisor decisions, corrections and evidence review history

Suggested fields to preserve per evidence record

- source_type: DEVICE_GEOLOCATION, IMAGE_EXIF, USER_SELECTED_PIN, FACILITY_REFERENCE, ADMIN_CORRECTION.
- latitude and longitude as decimal degrees; validate latitude from -90 to 90 and longitude from -180 to 180.
- accuracy_meters (nullable for EXIF because EXIF often doesn't provide a useful uncertainty radius).
- observed_at (source-provided timestamp) and received_at (backend timestamp).
- source_file_id or image hash if needed for evidence linkage; avoid duplicating the entire image in metadata.
- status, reason_codes, rule_version and reviewer decision for auditability.

Avoid saving raw EXIF dumps indefinitely. Extract only fields needed for the feature, restrict access, and keep uploaded media in controlled storage.

9. API contracts and example payloads

Illustrative endpoint design; names can be adjusted to match the project's API conventions.

Method and path	Purpose
POST /api/v1/incidents	Create complaint with selected incident location and optional device-location evidence; accept image upload via multipart or a separate media endpoint
POST /api/v1/incidents/{incident_id}/location-check	Run or rerun the incident-location comparison
POST /api/v1/tasks/{task_id}/completion-evidence	Submit after photo, device GPS and task completion data
GET /api/v1/incidents/{incident_id}/location-evidence	Authorized supervisor view of evidence and checks
POST /api/v1/location-checks/{check_id}/review	Supervisor records a review decision and reason

Example request schema (simplified)

```
{
  "description": "The public toilet is overflowing",
  "reported_location": {
    "latitude": 23.2500,
    "longitude": 77.4000,
    "source": "USER_SELECTED_PIN"
  },
  "device_location": {
    "latitude": 23.2503,
    "longitude": 77.4002,
    "accuracy_meters": 15.0,
    "observed_at": "2026-10-12T07:10:00Z",
    "source": "DEVICE_GEOLOCATION"
  },
  "media_ids": ["media_123"]
}
```

Illustrative response

```
{
  "status": "CONSISTENT",
  "distance_meters": 39.2,
  "target_source": "USER_SELECTED_PIN",
  "evidence_sources": ["DEVICE_GEOLOCATION"],
  "reason_codes": ["WITHIN_CONFIGURED_RADIUS"],
  "requires_human_review": false,
  "warning": "Location consistency is not proof that the complaint is true."
}
```

The coordinates and distance above are illustrative. Actual output depends on the input coordinates and configured policy.

API requirements

- Use authenticated identity for workers and supervisors. Citizen anonymous reporting policy should be explicit.
- Do not accept client-supplied status as authoritative; calculate status on the server.
- Use idempotency/request IDs for retries where duplicate submission is a risk.
- Apply upload size limits, MIME/type checks, image decoding safeguards and rate limits.
- Return reason codes that the UI can explain without exposing internal security rules.

10. Implementation examples

10.1 React/browser location capture

This is a minimal example for learning. Production code should handle loading state, permission denial, timeout, cancellation and user-facing explanation.

```
function getDeviceLocation() {
  return new Promise((resolve, reject) => {
    if (!navigator.geolocation) {
      reject(new Error("Geolocation is not supported."));
      return;
    }

    navigator.geolocation.getCurrentPosition(
      (position) => {
        resolve({
          latitude: position.coords.latitude,
          longitude: position.coords.longitude,
          accuracy_meters: position.coords.accuracy,
          observed_at: new Date(position.timestamp).toISOString(),
          source: "DEVICE_GEOLOCATION"
        });
      },
      (error) => reject(error),
      {
        enableHighAccuracy: true,
        timeout: 10000,
        maximumAge: 0
      }
    );
  });
}
```

10.2 EXIF GPS extraction outline with Pillow

```
from io import BytesIO
from PIL import Image, ExifTags

def read_gps_ifd(image_bytes: bytes) -> dict:
    # Add upload size/type checks before this function in production.
    with Image.open(BytesIO(image_bytes)) as image:
        exif = image.getexif()
        gps_ifd = exif.get_ifd(ExifTags.IFD.GPSInfo)
        named_tags = {
            ExifTags.GPSTAGS.get(tag_id, tag_id): value
            for tag_id, value in gps_ifd.items()
        }
        return {
            "has_gps_metadata": bool(named_tags),
            "gps_metadata": named_tags,
        }
```

This only reads raw GPS metadata. A separate tested function must convert DMS rational values to decimal degrees and apply N/S and E/W references. Catch malformed-image and metadata exceptions. Pillow documents access to the GPS IFD via get_ifd(ExifTags.IFD.GPSInfo). [3]

10.3 Distance calculation

```
from math import radians, sin, cos, asin, sqrt

def haversine_meters(lat1, lon1, lat2, lon2):
    # Validate coordinate ranges before calling this function.
    radius_m = 6_371_000
    lat1, lon1, lat2, lon2 = map(
        radians, (lat1, lon1, lat2, lon2)
    )
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = (
        sin(dlat / 2) ** 2
        + cos(lat1) * cos(lat2) * sin(dlon / 2) ** 2
    )
    a = min(1.0, max(0.0, a))  # protect against floating-point drift
    return 2 * radius_m * asin(sqrt(a))
```

10.4 Pydantic contracts

```
from datetime import datetime
from pydantic import BaseModel, Field

class Coordinates(BaseModel):
    latitude: float = Field(ge=-90, le=90)
    longitude: float = Field(ge=-180, le=180)

class DeviceLocation(Coordinates):
    accuracy_meters: float | None = Field(default=None, ge=0)
    observed_at: datetime | None = None
    source: str = "DEVICE_GEOLOCATION"

class LocationCheckResponse(BaseModel):
    status: str
    distance_meters: float | None
    reason_codes: list[str]
    requires_human_review: bool
```

Use explicit enums for status/source in the real project, and ensure timestamp timezone handling is consistent. Pydantic validation checks shape/ranges; it does not prove the submitted values are genuine.

11. Security, privacy and anti-spoofing limits

Location data can reveal sensitive movement patterns. KAVACH should collect only what is necessary for the task, explain the purpose, restrict access and define retention. Avoid routine logging of exact coordinates or complete EXIF metadata.

Controls to implement

- Consent and transparency: explain why location is requested and whether it is optional or required for that workflow.
- Least privilege: citizens see their own report; workers see assigned tasks; supervisors see evidence needed for review; admins have controlled access.
- Transport and secrets: HTTPS in deployment, secure authentication, no API keys in frontend, no secrets in Git.
- Validation: reject NaN/infinite values, invalid ranges, absurd accuracy values and malformed timestamps; cap upload size.
- Evidence provenance: store the source type, receipt time, task/incident ID and the rule version used.
- Abuse controls: rate limiting, duplicate submissions, audit logs and human review for suspicious patterns.
- Privacy: minimize retention, redact location from ordinary application logs and do not expose precise coordinates publicly.
- Human-safe decisions: a location mismatch is a signal, not an automatic accusation or penalty.

What anti-spoofing can and cannot do

A browser can submit modified coordinates; some devices permit mock locations; EXIF metadata can be edited; and two location signals may share the same underlying device or attacker-controlled source. Comparing signals increases confidence but does not make the system tamper-proof.

For higher assurance later

- Use a native mobile app if the product genuinely needs stronger device-level controls; still treat device attestation as a risk signal, not absolute proof.
- Require a task-specific QR code/NFC tag placed at the facility, or a rotating on-site code, for high-assurance worker check-in where operationally appropriate.
- Capture fresh before/after images in the task flow and bind them to task IDs and server receipt times.
- Use visual verification to inspect the facility/cleanliness state, while acknowledging that visual models can err.
- Use supervisor audits and random spot checks. Avoid collecting continuous location when a single task-time capture is enough.

12. Testing and acceptance criteria

The location module should be testable without Gemini, without live GPS and without a real PostgreSQL instance. Unit tests can use known coordinate pairs and mock evidence records.

Test case	Expected behaviour
Device GPS near the reported point, acceptable accuracy	Return CONSISTENT if configured policy is met; preserve distance and accuracy
Device GPS far from incident point	Return MISMATCH or REVIEW_REQUIRED, not 'fake complaint proven'
User denies location permission	UI offers fallback; backend records evidence unavailable if request continues
Geolocation timeout / unsupported browser	Graceful error state; no crash
GPS accuracy very poor	Return INCONCLUSIVE or review; do not treat the point as exact
EXIF has no GPS	Record EXIF_GPS_MISSING; continue other checks
EXIF coordinates are in DMS	Convert to decimal degrees correctly; test N/S and E/W
Malformed or oversized image	Reject safely or record extraction failure; server remains stable
EXIF GPS conflicts with device GPS	Record SOURCE_CONFLICT and request review according to policy
Invalid coordinates, NaN or infinity	Pydantic/service validation rejects input
Two toilets are very close	Do not claim GPS uniquely identifies one toilet; use facility-specific signal or manual review
Old photo with fresh device GPS	Do not assume the photo was taken at submission time; flag temporal evidence limitation
Repeated complaints in same area	Duplicate Detection may suggest a duplicate using distance plus content similarity
No Gemini API key in test environment	All location-service tests still pass in mock/offline mode
Unauthorized user requests evidence	API denies access and audit/security tests pass

Acceptance criteria for a first implementation

- Both citizen submission and worker completion can carry optional device location evidence.
- Image upload path can extract EXIF GPS where present and safely handle absent/invalid metadata.
- Distance is calculated server-side and the result contains status, distance, sources and reason codes.
- Source values are stored separately; one source never silently overwrites another.
- Missing/denied/inaccurate location does not crash the workflow.
- Agent inputs receive a structured location result rather than raw untrusted EXIF dumps.
- Tests cover positive, negative, missing, inaccurate, conflicting and malformed cases.
- Documentation explains that GPS consistency is not proof of authenticity.

13. Repository layout and implementation sequence

Suggested structure inside the FastAPI repository

```
src/kavach/
  api/
    routes/
      incidents.py
      task_evidence.py
      location_checks.py
  schemas/
    location.py
    incident.py
    task.py
  services/
    location/
      capture_contracts.py
      exif_reader.py
      coordinate_conversion.py
      distance.py
      verification.py
      rules.py
  agents/
    incident_intelligence.py
    duplicate_detection.py
    supervisor.py
    sanitation.py
    dispatch.py
    resolution_verification.py
  repositories/
    incident_repository.py
    location_evidence_repository.py
  models/
    incident.py
    facility.py
    task.py
    location_evidence.py
  tests/
    unit/
      test_distance.py
      test_exif_reader.py
      test_location_rules.py
    integration/
      test_incident_location_flow.py
      test_worker_evidence_flow.py
```

Build sequence

1. Contracts first. Define source enums, coordinate schemas, status enums and reason codes. Commit these before multiple developers build against different assumptions.

2. Pure Python service. Implement coordinate validation, distance calculation and decision rules. Write unit tests before connecting APIs.

3. Browser capture. Add an explicit 'Use my current location' action, permission/error handling and a visible accuracy indicator.

4. Image metadata. Implement safe Pillow inspection and DMS-to-decimal conversion; test files with GPS, without GPS and malformed EXIF.

5. FastAPI integration. Accept evidence, run checks server-side and return structured responses.

6. Agent integration. Pass a compact LocationEvidenceSummary to Incident Intelligence, Supervisor and Resolution Verification.

7. Persistence. Store evidence and checks in PostgreSQL; add migrations and access rules.

8. End-to-end tests. Test citizen report and worker completion flows using synthetic facility/worker data.

9. Field calibration. Measure real device accuracy around the target sites and adjust per-site policies before relying on flags operationally.

14. Prototype now vs later releases

Capability	First prototype	Later/production
Browser/device GPS	Yes, permission-based, optional fallback	Improve native support and field UX if needed
EXIF GPS	Yes, parse when present	Harden media pipeline and retention policies
Distance + rules	Yes, deterministic Python + tests	Calibrate thresholds and optionally use PostGIS
Citizen remote report	Allowed; location is a trust signal	Risk scoring with transparent review policy
Worker location evidence	Capture at task submission/completion	On-site QR/NFC/rotating code for selected sites
Gemini/LLM	Optional for complaint structuring; not required for location tests	Multimodal evidence support with evaluation and guardrails
PostgreSQL	Use when persistent records are needed; mock/in-memory for early isolated tests	Production migrations, indexes, backup and retention
Duplicate/hotspot analytics	Basic proximity signal may be mocked	Spatial indexing, aggregated maps and recurrence analytics
Fraud detection	Rule-based flags only	Evidence fusion and reviewed risk model if justified by data
Continuous tracking	No	Only if a separate, justified policy ever requires it

Recommended initial decision: build the Location Verification Service as a deterministic, independently testable module. Do not call it a 'fake complaint detector' yet; call it location consistency verification. It is accurate, explainable and easier to defend.

15. Glossary and references

Term	Plain meaning
Latitude / longitude	Coordinates representing a geographic point.
GPS / device geolocation	A device/browser-provided estimate of current location; it may use GPS, Wi-Fi, cellular and other signals depending on device/browser.
Accuracy (metres)	The device/browser's estimate of uncertainty around the reported position; not a guarantee.
EXIF	Metadata embedded in some image files, potentially including camera details, capture time and GPS.
Geofence / radius rule	A configurable distance rule around a target location.
Haversine distance	An approximate great-circle distance between two latitude/longitude points.
PostGIS	A PostgreSQL extension for geographic/spatial data and queries.
Provenance	A record of where evidence came from, when it was received and how it was used.
Evidence fusion	Combining independent signals instead of trusting a single one.
Inconclusive	Available evidence is not strong enough to support either a match or mismatch.

Technical references

[1] MDN Web Docs — Geolocation: getCurrentPosition(). https://developer.mozilla.org/en-US/docs/Web/API/Geolocation/getCurrentPosition

[2] MDN Web Docs — Geolocation API. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

[3] Pillow documentation — Image module / EXIF and get_ifd(). https://pillow.readthedocs.io/en/stable/reference/Image.html

[4] PostGIS documentation — ST_DWithin for radius queries. https://postgis.net/documentation/tips/st-dwithin/

Final engineering principle

KAVACH should store evidence, evaluate consistency, explain uncertainty and route ambiguous cases to a human. A location point is a useful signal—not a verdict.